<u>Εργασία 1 Τεχνητή Νοημοσύνη και Δίκτυα Υπολογιστών</u>

Στέργιος Μουτζίκος Inf2021149

Αρχικά εισάγω τις απαραίτητες βιβλιοθήκες. Αναλυτικά:

1. **import numpy as np**:
    a. Η βιβλιοθήκη NumPy χρησιμοποιείται για αριθμητικούς υπολογισμούς με πίνακες και διανύσματα. Χρησιμοποιώ τη NumPy για να δημιουργήσω τυχαίες θέσεις κόμβων σε έναν δισδιάστατο χώρο. Η συντόμευση np διευκολύνει την αναφορά στη NumPy σε όλο τον κώδικα.

2. **import networkx as nx**:
    a. Η βιβλιοθήκη NetworkX παρέχει εργαλεία για τη δημιουργία και διαχείριση γράφων (networks). Χρησιμοποιείται εδώ για τη δημιουργία του δικτύου (γράφου) των κόμβων και των ακμών του, καθώς και για την εύρεση των διαδρομών μεταξύ των κόμβων με διαφορετικούς αλγορίθμους.
    b. Με τη σύντμηση nx, οι συναρτήσεις της NetworkX μπορούν να χρησιμοποιηθούν εύκολα (π.χ., nx.Graph() για τη δημιουργία ενός γράφου).

3. **import matplotlib.pyplot as plt**:
    a. Η matplotlib.pyplot είναι μια βιβλιοθήκη για σχεδιασμό γραφημάτων. Εδώ χρησιμοποιείται για να εμφανίσει τα αποτελέσματα του κώδικα σε μορφή γραφήματος μπάρας, ώστε να συγκριθούν οι αλγόριθμοι.

4. **from scipy.spatial.distance import euclidean**:

a. Η scipy.spatial.distance είναι μια υποενότητα της βιβλιοθήκης SciPy, και η euclidean είναι συνάρτηση που υπολογίζει την Ευκλείδεια απόσταση μεταξύ δύο σημείων. Εδώ χρησιμοποιείται για να μετρήσει την απόσταση μεταξύ κόμβων, προκειμένου να καθοριστεί αν υπάρχει ακμή (σύνδεση) μεταξύ τους, βάσει της ακτίνας rc, καθώς και στην ευρετική συνάρτηση του A*.

5. **import time**:
   a. time: Η βιβλιοθήκη time χρησιμοποιείται για να μετράει τον χρόνο εκτέλεσης, ώστε να καταγράφεται ο χρόνος που απαιτεί κάθε αλγόριθμος για να υπολογίσει τις διαδρομές.

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from scipy.spatial.distance import euclidean
import time
```

Έπειτα, καθορίζονται κάποιες βασικές παράμετροι: num_nodes (ο αριθμός των κόμβων του δικτύου, εδώ 50), area_size (το μέγεθος της περιοχής όπου θα τοποθετηθούν οι κόμβοι, δηλαδή 1000x1000 μονάδες) και rc (η αρχική ακτίνα σύνδεσης, που ξεκινά από 0 και θα αυξάνεται σε επόμενο στάδιο για να επιτευχθεί συνδεσιμότητα). Με τη χρήση της np.random.rand(num_nodes, 2) * area_size, δημιουργούνται τυχαίες θέσεις για τους κόμβους μέσα στον δισδιάστατο χώρο, με τιμές μεταξύ 0 και 1000.

Η συνάρτηση create_network(rc) χρησιμοποιείται για τη δημιουργία ενός δικτύου (γράφου) με βάση μια συγκεκριμένη ακτίνα σύνδεσης rc. Αρχικά, δημιουργείται ένας κενός γράφος G και προστίθενται σε αυτόν όλοι οι κόμβοι, καθένας από τους οποίους συνοδεύεται από τις συντεταγμένες της θέσης του στον χώρο. Έπειτα, δύο εμφωλευμένοι βρόχοι συγκρίνουν κάθε ζεύγος κόμβων. Για κάθε ζεύγος κόμβων i και j, υπολογίζεται η Ευκλείδεια απόσταση μεταξύ τους με τη συνάρτηση euclidean. Αν η απόσταση είναι μικρότερη ή ίση από την ακτίνα rc, τότε προστίθεται μια ακμή (σύνδεση) μεταξύ των κόμβων i και j στον γράφο G, με βάρος ίσο με την απόσταση αυτή.

Τέλος, η συνάρτηση επιστρέφει τον γράφο G μόνο εάν είναι συνδεδεμένος, δηλαδή εάν υπάρχει ένα μονοπάτι μεταξύ κάθε ζεύγους κόμβων. Αν ο γράφος δεν είναι συνδεδεμένος,

η συνάρτηση επιστρέφει None, υποδεικνύοντας ότι η ακτίνα rc δεν ήταν αρκετή για να συνδέσει όλους τους κόμβους.

```
# Parameters
num_nodes, area_size, rc = 50, 1000, 0
positions = np.random.rand(num_nodes, 2) * area_size

# Function to create the network with radius rc
Codeium: Refactor | Explain | Generate Docstring | X
def create_network(rc):
    G = nx.Graph()
    for i in range(num_nodes):
        G.add_node(i, pos=positions[i])

    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
            dist = euclidean(positions[i], positions[j])
            if dist <= rc:
                G.add_edge(i, j, weight=dist)
    return G if nx.is_connected(G) else None
```

Στη συνέχεια κατασκευάζω ένα συνδεδεμένο δίκτυο κόμβων αυξάνοντας σταδιακά την ακτίνα σύνδεσης rc μέχρι το δίκτυο να συνδεθεί πλήρως. Στη συνέχεια, χρησιμοποιώ τους τρεις αλγόριθμους (BFS, Dijkstra, και A*) για τον υπολογισμό των μονοπατιών μεταξύ των κόμβων και εξάγω στατιστικά σχετικά με τις διαδρομές και τον χρόνο εκτέλεσης.

Αναλυτικότερα, το πρώτο μέρος του κώδικα είναι ένας βρόχος while, ο οποίος καλεί τη συνάρτηση create_network(rc) με μια αυξανόμενη τιμή ακτίνας σύνδεσης rc, ξεκινώντας από 0 και αυξάνοντάς τη κατά 10 μονάδες σε κάθε επανάληψη. Ο στόχος είναι να διασφαλιστεί ότι όλοι οι κόμβοι θα συνδεθούν σε ένα ενιαίο δίκτυο. Ο βρόχος συνεχίζεται μέχρι ο γράφος G που επιστρέφει η create_network να είναι συνδεδεμένος. Μόλις επιτευχθεί συνδεσιμότητα, ο βρόχος τερματίζεται.

Το δεύτερο μέρος είναι η συνάρτηση calculate_paths(graph, algorithm), στην οποία υπολογίζω τις αποστάσεις και το χρόνο εκτέλεσης για μονοπάτια μεταξύ όλων των ζευγών κόμβων του δικτύου χρησιμοποιώντας τον αλγόριθμο που καθορίζεται στη μεταβλητή algorithm. Για κάθε ζεύγος κόμβων (i, j), η συνάρτηση:

1. Καταγράφει την έναρξη του χρόνου εκτέλεσης.
2. Ανάλογα με τον επιλεγμένο αλγόριθμο, υπολογίζει τη διαδρομή:

a. **BFS** (χωρίς βάρη): Υπολογίζει το μικρότερο μονοπάτι χωρίς να λαμβάνει υπόψη τα βάρη των ακμών. Μετρά την απόσταση συνοψίζοντας τα βάρη των ακμών του μονοπατιού.
b. **Dijkstra**: Υπολογίζει το μικρότερο μονοπάτι με βάρη. Η nx.dijkstra_path βρίσκει το μονοπάτι ελάχιστου βάρους, ενώ η nx.path_weight υπολογίζει το συνολικό βάρος.
c. **A***: Χρησιμοποιεί Ευκλείδεια απόσταση ως ευρετική για να επιταχύνει τον υπολογισμό του μικρότερου μονοπατιού με βάρη.

Για κάθε μονοπάτι καταγράφονται: η απόσταση (length), ο αριθμός των βημάτων (κόμβοι που διέρχεται το μονοπάτι) και ο χρόνος εκτέλεσης. Τέλος, η συνάρτηση επιστρέφει τους μέσους και διάμεσους όρους των αποστάσεων, τον μέσο αριθμό βημάτων και τον μέσο χρόνο εκτέλεσης για το επιλεγμένο αλγόριθμο.

```python
# Create a connected graph with increasing radius rc until connectivity is ensured
while not (G := create_network(rc := rc + 10)):
    pass

# Function to calculate paths and statistics
Codeium: Refactor | Explain | Generate Docstring | X
def calculate_paths(graph, algorithm):
    path_lengths, num_steps, times = [], [], []
    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
            start_time = time.time()

            # Choose algorithm and calculate path
            if algorithm == 'bfs':  # BFS without weights
                path = nx.shortest_path(graph, i, j)
                length = sum(graph[u][v]['weight'] for u, v in zip(path, path[1:]))
            elif algorithm == 'dijkstra':  # Dijkstra with weights
                path = nx.dijkstra_path(graph, i, j, weight='weight')
                length = nx.path_weight(graph, path, weight='weight')
            else:  # A* with Euclidean heuristic
                path = nx.astar_path(graph, i, j, heuristic=lambda u, v: euclidean(positions[u], positions[v]), weight='weight')
                length = nx.path_weight(graph, path, weight='weight')

            path_lengths.append(length)
            num_steps.append(len(path) - 1)  # Number of steps in path
            times.append(time.time() - start_time)

    # Return measurements: average distance, median distance, average steps, average time
    return np.mean(path_lengths), np.median(path_lengths), np.mean(num_steps), np.mean(times)
```

Έπειτα υπολογίζω τα στατιστικά στοιχεία για τους τρεις αλγόριθμους (BFS, Dijkstra και A*) και οπτικοποιώ τα αποτελέσματα σε γραφήματα. Αρχικά, δημιουργείται ένα λεξικό results, όπου κλειδιά είναι τα ονόματα των αλγορίθμων και οι τιμές είναι τα αποτελέσματα από τη συνάρτηση calculate_paths(G, alg), η οποία υπολογίζει τις αποστάσεις και τον χρόνο εκτέλεσης για κάθε αλγόριθμο.

Με τη χρήση της συνάρτησης zip, τα αποτελέσματα αποσυναρμολογούνται σε τέσσερις μεταβλητές: avg_distances, median_distances, avg_steps, και avg_times, οι οποίες

περιέχουν αντίστοιχα τις μέσες αποστάσεις, τις διάμεσες αποστάσεις, τον μέσο αριθμό βημάτων και τον μέσο χρόνο εκτέλεσης για τους τρεις αλγόριθμους.

Στη συνέχεια, αρχίζει η διαδικασία σχεδίασης των αποτελεσμάτων. Δημιουργούνται υπογράμματα (axes) σε ένα μόνο γράφημα με τη χρήση της plt.subplots, ορίζοντας 1 σειρά και 3 στήλες με μέγεθος 18x6. Οι αλγόριθμοι (BFS, Dijkstra, A*) αποθηκεύονται σε μια λίστα algorithms, έτοιμοι για να χρησιμοποιηθούν στην οπτικοποίηση των στατιστικών αποτελεσμάτων.

```
# Calculations for each algorithm
results = {alg: calculate_paths(G, alg) for alg in ['bfs', 'dijkstra', 'astar']}
avg_distances, median_distances, avg_steps, avg_times = zip(*results.values())

# Plotting results
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
algorithms = ['BFS', 'Dijkstra', 'A*']
```

Τέλος, δημιουργώ γραφήματα για να απεικονίσω τις μέσες αποστάσεις, τον αριθμό βημάτων και τον χρόνο εκτέλεσης των τριών αλγορίθμων (BFS, Dijkstra και A*). Χρησιμοποιώντας τη συνάρτηση zip, η οποία συνδυάζει τα υπογράμματα, τα δεδομένα και τις ετικέτες για κάθε γράφημα, ξεκινά ένας βρόχος που επαναλαμβάνει για κάθε υπογράφημα (ax), δεδομένα (data), τίτλο (title) και ετικέτα άξονα (ylabel).

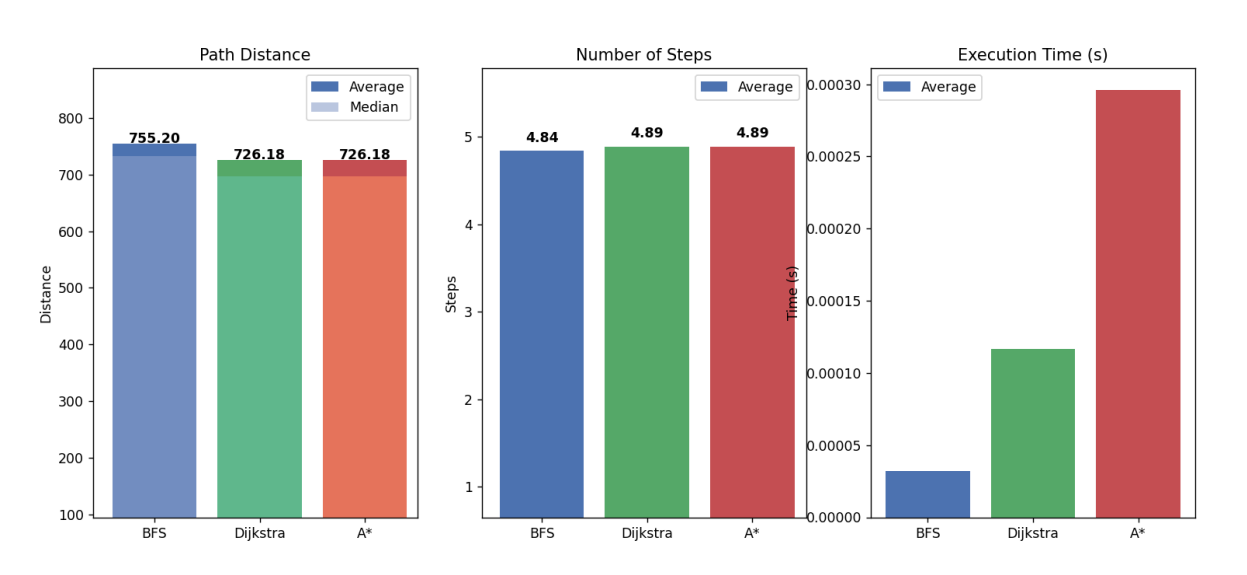
Στο εσωτερικό του βρόχου, το πρώτο γράφημα σχεδιάζεται με τις μέσες αποστάσεις ή τα βήματα ή τους χρόνους εκτέλεσης, χρησιμοποιώντας τη μέθοδο ax.bar για να δημιουργήσει ένα ραβδόγραμμα, με διαφορετικά χρώματα για κάθε αλγόριθμο. Αν υπάρχουν περισσότερες από μία τιμές (δηλαδή σε περίπτωση που σχεδιάζονται οι διάμεσες τιμές), τότε προστίθεται ένα δεύτερο ραβδόγραμμα για τις διάμεσες τιμές, με μειωμένη διαφάνεια (alpha=0.6) ώστε να ξεχωρίζουν τα δύο σύνολα δεδομένων.

Κατόπιν, οι τίτλοι και οι ετικέτες του άξονα ορίζονται μέσω της ax.set(), και προστίθεται μια υπογραφή (legend) για να διευκρινίζεται ποια μπάρες αντιπροσωπεύουν τις μέσες και ποιες τις διάμεσες τιμές. Τέλος, για κάθε μπάρα του ραβδόγραμμα, η μέση τιμή τοποθετείται πάνω από την αντίστοιχη μπάρα, με χρήση της ax.text(), παρέχοντας έτσι οπτική ανατροφοδότηση στον θεατή.

Αφού ολοκληρωθούν όλα τα υπογράμματα, η plt.tight_layout() καλείται για να εξασφαλίσει ότι τα υπογράμματα και οι ετικέτες δεν επικαλύπτονται και, τέλος, η plt.show() καλεί το Matplotlib να εμφανίσει τα γραφήματα στην οθόνη.

```python
# Graph for distances (Average and Median)
for ax, data, title, ylabel in zip(
        axes, [(avg_distances, median_distances), (avg_steps,), (avg_times,)],
        ["Path Distance", "Number of Steps", "Execution Time (s)"], ["Distance", "Steps", "Time (s)"]):

    bars1 = ax.bar(algorithms, data[0], color=['#4c72b0', '#55a868', '#c44e52'], label='Average')

    # Median values if applicable for distances
    if len(data) > 1:
        bars2 = ax.bar(algorithms, data[1], color=['#8da0cb', '#66c2a5', '#fc8d62'], alpha=0.6, label='Median')

    # Titles and labels
    ax.set(title=title, ylabel=ylabel)
    ax.legend()

    # Display values above bars
    for bar, avg in zip(bars1, data[0]):
        offset = 0.5 if title == "Path Distance" else 0.1
        ax.text(bar.get_x() + bar.get_width() / 2, avg + offset, f"{avg:.2f}", ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```

Το διάγραμμα:



---> 1. Απόσταση Διαδρομής (Path Distance)

Στο πρώτο διάγραμμα, βλέπουμε τις μέσες και διάμεσες αποστάσεις διαδρομής για κάθε αλγόριθμο:

BFS: Ο αλγόριθμος BFS, ο οποίος δεν λαμβάνει υπόψη βάρη ακμών, εμφανίζει μεγαλύτερη μέση απόσταση (755.20), επειδή υπολογίζει διαδρομές με βάση τον αριθμό βημάτων και όχι το πραγματικό μήκος.

Dijkstra και A*: Οι αλγόριθμοι Dijkstra και A*, οι οποίοι λαμβάνουν υπόψη τις αποστάσεις των ακμών, παράγουν μικρότερες αποστάσεις (726.18) που είναι παρόμοιες μεταξύ τους. Αυτό συμβαίνει γιατί και οι δύο επιλέγουν τις βέλτιστες διαδρομές με βάση την απόσταση, με το A* να είναι ελαφρώς ταχύτερο στις υπολογιστικές αποστάσεις.

---> 2. Αριθμός Βημάτων (Number of Steps)

Στο δεύτερο διάγραμμα εμφανίζεται ο μέσος αριθμός βημάτων που απαιτείται για κάθε αλγόριθμο:

Όλοι οι αλγόριθμοι παρουσιάζουν παρόμοιο αριθμό βημάτων, περίπου 4.84 έως 4.89. Αυτό δείχνει ότι, ενώ η απόσταση μπορεί να ποικίλει ανάλογα με τον αλγόριθμο, ο αριθμός βημάτων για τη σύνδεση των κόμβων παραμένει συγκριτικά σταθερός. Οι διαφορές στον αριθμό βημάτων είναι ελάχιστες, κάτι που είναι αναμενόμενο σε ένα τυχαίο δίκτυο κόμβων.

---> 3. Χρόνος Εκτέλεσης (Execution Time)

Στο τρίτο διάγραμμα, παρουσιάζεται ο μέσος χρόνος εκτέλεσης για κάθε αλγόριθμο:

BFS: Ο BFS έχει τον μικρότερο χρόνο εκτέλεσης, επειδή δεν χρησιμοποιεί βάρη ακμών.

Dijkstra: Παρουσιάζει μέσο χρόνο εκτέλεσης υψηλότερο από το BFS, αφού λαμβάνει υπόψη βάρη ακμών και υπολογίζει βέλτιστες διαδρομές.

A*: Έχει τον μεγαλύτερο χρόνο εκτέλεσης. Ο λόγος είναι η χρήση ευρετικής συνάρτησης (Ευκλείδειας απόστασης), η οποία αυξάνει την πολυπλοκότητα του αλγορίθμου.

---> Συμπεράσματα

Από τα διαγράμματα αυτά, συμπεραίνουμε:

Ο αλγόριθμος BFS είναι ο ταχύτερος, αλλά παράγει μεγαλύτερες αποστάσεις διαδρομής λόγω της απουσίας βαρών.

Οι αλγόριθμοι Dijkstra και A* επιτυγχάνουν μικρότερες αποστάσεις διαδρομής, με τον Dijkstra να είναι πιο αποδοτικός από τον A* όσον αφορά το χρόνο, ενώ ο A* είναι πιο ακριβής λόγω της χρήσης ευρετικής.